The Evolution of AMSI Evasion: From Memory Patching to Hardware Breakpoints

1. Introduction

Fileless execution is a staple of modern intrusion operations. Instead of dropping a portable executable to disk, attackers pipe payloads directly into trusted Windows binaries like `powershell.exe`, `wscript.exe`, or `mshta.exe`. This leaves no traditional file artifacts for antivirus software to scan. The Windows Antimalware Scan Interface (AMSI) was introduced in Windows 10 to close this gap.

AMSI provides a standardized API that allows host applications to submit in-memory content to a registered antivirus provider before execution. If Windows Defender flags the buffer, the host application blocks the script. The implementation forces attackers to heavily obfuscate payloads or find ways to disable the scanning engine entirely. The latter approach relies on a structural weakness in how Microsoft integrated AMSI into the Windows architecture.

2. The Architectural Flaw

To understand why AMSI is routinely bypassed, look at where the scanning logic actually runs. AMSI is not a protected, kernel-mode service. The `amsi.dll` library is loaded directly into the user-mode process that requires scanning. When a user runs a PowerShell script, `amsi.dll` is mapped into the memory space of `powershell.exe`.

The host application executes a sequence of API calls to perform a scan. It initializes the interface with `AmsiInitialize`, opens a session with `AmsiOpenSession`, and submits the script content using `AmsiScanBuffer`. This function passes the buffer to the registered antivirus provider, waits for a response, and returns an HRESULT to the host. A return value of `0` signifies `AMSI_RESULT_CLEAN`, while `32768` (`0x8000`) means the content is flagged as malicious.

Because `amsi.dll` operates within the same address space as the script it is inspecting, it shares the same privilege level. The application running the script has the necessary permissions to read and write to the memory regions where `amsi.dll` resides. If an attacker controls the execution flow of the host process, they can manipulate the AMSI functions directly. The security mechanism relies on the host application trusting a library that runs inside an environment the attacker already controls. The guard is locked in the same room as the attacker, and the attacker holds the keys.

3. Early Evasions and Memory Patching

Before attackers started writing complex memory patches, they looked for configuration weaknesses or host-level flaws.

Downgrading to PowerShell 2.0 remains the simplest bypass. The older engine predates AMSI integration. Launching `powershell.exe -version 2` drops the operator into a scripting environment where AMSI does not exist, completely bypassing the scan.

Other early methods targeted the AMSI provider registration. The antivirus engine registers its AMSI provider via a GUID under `HKLM:\SOFTWARE\Microsoft\AMSI\Providers\{2781761E-28E0-4109-99FE-B9D127C57AFE}`. Deleting this key prevents `amsi.dll` from locating the provider. However, this requires administrative privileges and generates high-fidelity telemetry, making it operationally noisy.

DLL hijacking was another approach. If an attacker can place a malicious `amsi.dll` in a directory that precedes the System32 path in the DLL search order for a specific application, the application will load the fake library instead. SensePost demonstrated this by copying PowerShell to a user-writable directory and planting a proxy `amsi.dll` alongside it.

Matt Graeber moved away from configuration and targeted the host engine itself. His technique exploited the internal state of PowerShell. The engine maintains a class named `System.Management.Automation.AmsiUtils`. This class contains a static boolean field called `amsiInitFailed`. By default, it is set to false. If an attacker uses .NET reflection to change this field to true, PowerShell assumes the AMSI provider is broken and stops requesting scans altogether.

```
[Ref].Assembly.GetType('System.Management.Automation.AmsiUtils').GetField('amsiInitFailed','NonPublic,Stati
c').SetValue($null,$true)
```

To evade static signatures, attackers later used string concatenation or Base64 encoding. The strings `AmsiUtils` and `amsiInitFailed` were split and reassembled at runtime, preventing simple string-matching rules from flagging the script.

While the flag flip disables the engine, direct memory patching neutralizes the library. Daniel Duggan popularized this technique in C# and PowerShell. The goal is to force `AmsiScanBuffer` to return a clean result before it can execute its scanning logic.

The process requires three steps. First, `GetProcAddress` locates the memory address of `AmsiScanBuffer` inside the loaded `amsi.dll`. Second, the `VirtualProtect` API is called to change the memory protection of that address from read-only to `PAGE_EXECUTE_READWRITE`. Third, a small byte array is written over the function's prologue using `Marshal.Copy`.

Duggan's patch uses the following bytes:

```
static byte[] x64 = new byte[] { 0xB8, 0x57, 0x00, 0x07, 0x80, 0xC3 };
```

This payload translates to specific x86-64 assembly instructions. `0xB8` is the opcode for `MOV EAX`, meaning the CPU should move the following four bytes into the EAX register. The bytes `0x57, 0x00, 0x07, 0x80` represent the little-endian format of `0x80070057`, which is the hexadecimal code for `E_INVALIDARG` (Invalid Argument). The final byte, `0xC3`, is the opcode for `RET`, which returns execution to the caller.

When the host calls `AmsiScanBuffer`, the CPU executes the patched instructions. It immediately places the `E_INVALIDARG` error code into the return register and exits the function. The actual scanning code is never executed, and the host application treats the error as a clean result.

4. The Patchless Shift: Hardware Breakpoints and VEH

Modern Endpoint Detection and Response (EDR) platforms monitor the memory space of critical processes. When an attacker uses `VirtualProtect` and overwrites the prologue of `AmsiScanBuffer`, the EDR detects the unauthorized modification of the `amsi.dll` memory region. Memory patching became a noisy operation.

To bypass AMSI without triggering memory integrity checks, offensive researchers developed patchless techniques. These methods leave the `amsi.dll` code entirely intact and instead manipulate the CPU's execution state.

The most prominent method relies on hardware breakpoints and Vectored Exception Handlers (VEH). A hardware breakpoint uses the CPU's debug registers, specifically `DR0` through `DR3`, to monitor specific memory addresses.

An attacker writes the memory address of `AmsiScanBuffer` into the `DR0` register. When the host application calls `AmsiScanBuffer`, the CPU detects that the instruction pointer has reached the address stored in `DR0`. The CPU then raises a breakpoint exception before executing the function's code.

The attacker registers a custom Vectored Exception Handler using `AddVectoredExceptionHandler`. When the exception fires, the VEH examines the thread context. It alters the `RAX` register to contain the value `0` (which signifies `AMSI_RESULT_CLEAN`) and modifies the `RIP` (Instruction Pointer) register to point to the return address of the function. The handler then passes execution back to the CPU, which resumes running the host application as if `AmsiScanBuffer` had executed normally and returned a clean result.

```
// Inside the VEH exception handler
if (ExceptionRecord->ExceptionCode == EXCEPTION_SINGLE_STEP) {
    context->Rax = 0; // AMSI_RESULT_CLEAN
    context->Rip = (DWORD64)return_address; // Skip the function body
    return EXCEPTION_CONTINUE_EXECUTION;
}
```

Because the bytes inside `amsi.dll` are never altered, memory scanners find nothing. The bypass exists purely within the CPU's debug state and the process's exception handling mechanism.

A secondary challenge arises during the setup phase. Modifying debug registers typically requires calling the `SetThreadContext` API. Modern EDRs instrument this API through Event Tracing for Windows (ETW) Threat Intelligence, generating an `EtwTiLogSetContextThread` event that can reveal the manipulation of debug registers.

In 2025, researchers introduced variations such as the `VEH²` technique to address this telemetry. Instead of using `SetThreadContext`, this method uses `NtContinue`, a lower-level API typically used for exception handling, to modify the thread context and set the debug registers. This approach avoids generating the specific ETW TI event associated with thread context modification, allowing the hardware breakpoint to be set with minimal telemetry footprint. The attacker hijacks the execution thread without ever writing to the AMSI library or generating a high-level API telemetry event.

5. Conclusion

The history of AMSI evasion demonstrates a continuous cycle of defense and adjustment. Microsoft improved detection by monitoring memory integrity, and attackers responded by moving the manipulation out of memory and into the CPU's execution state.

The root cause of this cycle is the architectural placement of AMSI. Because `amsi.dll` operates in user mode inside the application being scanned, it relies on the integrity of an environment that the attacker can compromise. Any code running within that process can theoretically intercept, manipulate, or bypass the AMSI function calls. Until the scanning mechanism is moved outside the attacker-controlled address space, evasion techniques will continue to adapt around whatever runtime protections Microsoft implements. The shift from memory patching to hardware breakpoints illustrates that the security boundary is fundamentally flawed, requiring defenders to monitor lower-level execution states rather than just static memory modifications.